

Title: WAP to implement Caesar Cipher Algorithm.

Caesar Cipher:

Caesar Cipher is one of the oldest and simplest encryption algorithms. It is a type of monoalphabetic substitution cipher, meaning each letter in the plaintext is substituted with another letter. Each letter in the plaintext is replaced by a letter some fixed number of positions forward (for encryption) or backward (for decryption) in the alphabet. The shift value is known as the key.

Algorithm:

1. Start.
2. Input plaintext and key.
3. Convert each letter of the plaintext to its corresponding number (A = 0, B = 1, ..., Z = 25).
4. Encrypt each letter using: $C = (P + K) \bmod 26$, where P is the plaintext letter and K is the key.
5. Convert the resulting numbers back to letters to form the ciphertext.
6. To decrypt, use: $P = (C - K + 26) \bmod 26$, where C is the ciphertext letter.
7. Convert the resulting numbers back to letters to retrieve the original plain text.
8. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void caesar_encrypt(char* text, int shift, char* result) {
    int i = 0;
    while (text[i] != '\0') {
        if (isalpha(text[i])) {
            char base = isupper(text[i]) ? 'A' : 'a';
            result[i] = (base + (text[i] - base + shift) % 26);
        } else {
            result[i] = text[i];
        }
        i++;
    }
    result[i] = '\0';
}

void caesar_decrypt(char* text, int shift, char* result) {
    int i = 0;
    while (text[i] != '\0') {
        if (isalpha(text[i])) {
            char base = isupper(text[i]) ? 'A' : 'a';
            result[i] = (base + (text[i] - base - shift + 26) % 26);
        }
        i++;
    }
    result[i] = '\0';
}
```

```

        } else {
result[i] = text[i];
        }
i++;
result[i] = '\0';
} int main() {      char text[100], encrypted[100],
decrypted[100];    int shift;

printf("=====\n");
printf("      Caesar Cipher Program\n");
printf("=====\n");
    printf("Enter the text:
");      scanf("%s", text);
    printf("Enter the shift value (numeric key): ");
scanf("%d", &shift);
    caesar_encrypt(text, shift, encrypted);
printf("\nEncrypted Text: %s\n", encrypted);
    caesar_decrypt(encrypted,      shift,
decrypted);      printf("Decrypted Text: %s\n",
decrypted);
    return
0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
      Caesar Cipher Program
=====
Enter the text: HELLOSARFUU
Enter the shift value (numeric key): 2

Encrypted Text: JGNNQUCTHWW
Decrypted Text: HELLOSARFUU

-----
Process exited after 6.797 seconds with return value 0
Press any key to continue . . .

```

Title: WAP to implement Polyalphabetic Caesar Cipher Algorithm (Vigner's).

Vigenère Cipher:

The Vigenère Cipher, often referred to as a polyalphabetic Caesar Cipher, is a polyalphabetic substitution cipher that improves upon the monoalphabetic Caesar Cipher by using a keyword to apply multiple shift values for encryption. Each letter of the plaintext is shifted by the numerical value of the corresponding letter in a repeating keyword (A=0, B=1, ..., Z=25), making it more secure against frequency analysis attacks. The key is extended to match the plaintext length by repeating the keyword, and non-alphabetic characters in the plaintext remain unchanged during encryption and decryption.

Algorithm

1. Start.
2. Input plaintext and a keyword (key).
3. Extend the key to match the plaintext length by repeating the keyword, ignoring non-alphabetic characters in the plaintext.
4. Convert each letter of the plaintext and the extended key to its corresponding number (A=0, B=1, ..., Z=25).
5. Encrypt each plaintext letter using: $C_i = (P_i + K_i) \bmod 26$, where P_i is the i -th plaintext letter's numerical value, K_i is the i -th key letter's numerical value, and C_i is the resulting ciphertext letter.
6. Convert the resulting numbers back to letters to form the ciphertext, preserving case and non-alphabetic characters.
7. To decrypt, use: $P_i = (C_i - K_i + 26) \bmod 26$, where C_i is the i -th ciphertext letter's numerical value.
8. Convert the resulting numbers back to letters to retrieve the original plaintext, preserving case and non-alphabetic characters.
9. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int char_to_int(char c) {
    return tolower(c) - 'a';
}
void extend_key(const char* text, const char* key, char*
extended_key) {
    int key_len = strlen(key);
    int key_index = 0;
    int i;
    for (i = 0; text[i] != '\0'; i++) {
        if
```

```

(isalpha(text[i])) {          extended_key[i] = key[key_index %
key_len];          key_index++;          } else {
extended_key[i] = text[i];
    }    }
extended_key[strlen(text)] = '\0';
} void vigenere_encrypt(const char* text, const char* key, char*
result) {          char extended_key[100];          extend_key(text, key,
extended_key); int i;          for ( i = 0; text[i] != '\0'; i++) {
if (isalpha(text[i])) {          char base = isupper(text[i]) ? 'A'
: 'a';          int p = text[i] - base;          int k =
char_to_int(extended_key[i]);          result[i] = base + (p + k) %
26;

    } else {
result[i] = text[i];
    }    }
result[strlen(text)] = '\0';
} void vigenere_decrypt(const char* text, const char* key, char*
result) {          char extended_key[100];          extend_key(text, key,
extended_key); int i;          for ( i = 0; text[i] != '\0'; i++) {
if (isalpha(text[i])) {          char base = isupper(text[i]) ? 'A'
: 'a';          int c = text[i] - base;          int k =
char_to_int(extended_key[i]);          result[i] = base + (c - k +
26) % 26;

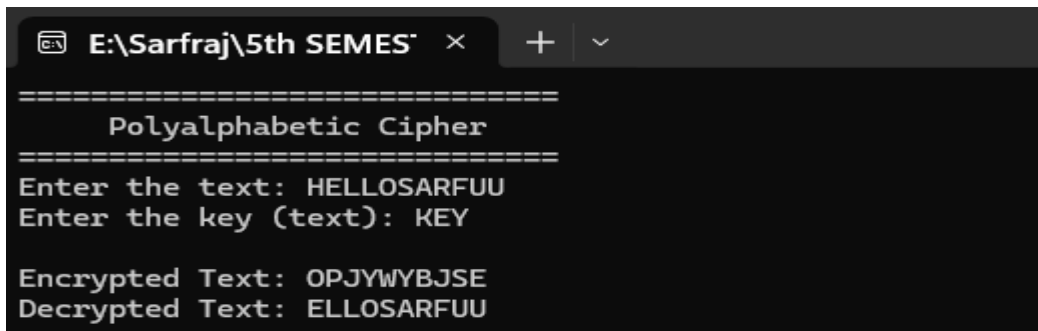
    } else {
result[i] = text[i];
    }    }
result[strlen(text)] = '\0';
}

int main() {          char text[100], key[100], encrypted[100],
decrypted[100];

printf("=====\n");
printf("    Polyalphabetic Cipher\n");
printf("=====\n");
    printf("Enter the text: ");          getchar(); // Clear input
buffer          fgets(text, sizeof(text), stdin);
text[strcspn(text, "\n")] = '\0'; // Remove trailing newline
    printf("Enter the key (text): ");
scanf("%s", key);
    vigenere_encrypt(text, key, encrypted);
printf("\nEncrypted Text: %s\n", encrypted);
    vigenere_decrypt(encrypted, key,
decrypted);          printf("Decrypted Text: %s\n",
decrypted);
    return
0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
=====
Polyalphabetic Cipher
=====
Enter the text: HELLOSARFUU
Enter the key (text): KEY

Encrypted Text: OPJYWYBJSE
Decrypted Text: ELLOSARFUU

```

Title: WAP to implement Playfair Cipher

Playfair Cipher

The Playfair Cipher is a digraph substitution cipher that encrypts pairs of letters (digraphs) using a 5x5 matrix constructed from a keyword. Unlike monoalphabetic ciphers like the Caesar Cipher, it processes two letters at a time, making it more secure against simple frequency analysis. The matrix is built by filling in the keyword (with duplicates removed and I/J treated as one letter) followed by the remaining alphabet letters (excluding J). During encryption, plaintext is divided into digraphs, with a filler letter 'X' added for identical letters in a pair or for odd-length text. Non-alphabetic characters remain unchanged. The encryption rules depend on the positions of the digraph letters in the matrix: same row, same column, or forming a rectangle. Decryption reverses these rules.

Algorithm

1. Start.
2. Input plaintext and keyword.
3. Generate a 5x5 matrix from the keyword (duplicates removed, I/J combined) and remaining alphabet (excluding J).
4. Form digraphs from plaintext: insert 'X' for identical letters or odd length, preserve non-alphabetic characters.
5. Encrypt each digraph using the matrix:
 - **Same row:** shift each letter right (wrap around).
 - **Same column:** shift each letter down (wrap around).
 - **Rectangle/ Diagonal:** swap each letter with the one in its row and the other's column.
6. Decrypt by reversing: shift left for same row, up for same column, same for rectangle.
7. Preserve letter case and non-alphabetic characters.
8. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void generate_matrix(const char *key, char matrix[5][5]) {
    int used[26] = {0};
    char key_clean[26];
    int k = 0, index = 0;
    for (int i = 0; key[i] != '\0'; i++) {
        char c = tolower(key[i]);
```

```

        if (c == 'j') c = 'i';
        if (isalpha(c) && !used[c - 'a']) {
            key_clean[k++] = c;
            used[c - 'a'] = 1;
        }
    }

    for (char c = 'a'; c <= 'z'; c++) {
        if (c == 'j') continue;
        if (!used[c - 'a']) {
            key_clean[k++] = c;
            used[c - 'a'] = 1;
        }
    }
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            matrix[i][j] = key_clean[index++];
        }
    }
}

void find_position(char c, char matrix[5][5], int *row, int *col) {
    if (c == 'j') c = 'i';
    c = tolower(c);

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (matrix[i][j] == c) {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

void process_digraphs(const char *text, char matrix[5][5], int encrypt, char *result) {
    char alpha_text[100] = "";
    int is_upper[100] = {0};
    int text_len = strlen(text), k = 0;
    for (int i = 0; i < text_len; i++) {
        if (isalpha(text[i])) {
            alpha_text[k] = tolower(text[i]);
            is_upper[k] = isupper(text[i]);
            k++;
        }
    }
    char digraphs[100] = "";
    int digraph_len = 0;
    for (int i = 0; i < k; i++) {
        digraphs[digraph_len++] = alpha_text[i];
        if (i + 1 < k && alpha_text[i] == alpha_text[i + 1]) {
            digraphs[digraph_len++] = 'x';
        } else if (i + 1 == k) {
            digraphs[digraph_len++] = 'x';
        } else {
            digraphs[digraph_len++] = alpha_text[i + 1];
        }
    }
}

```

```

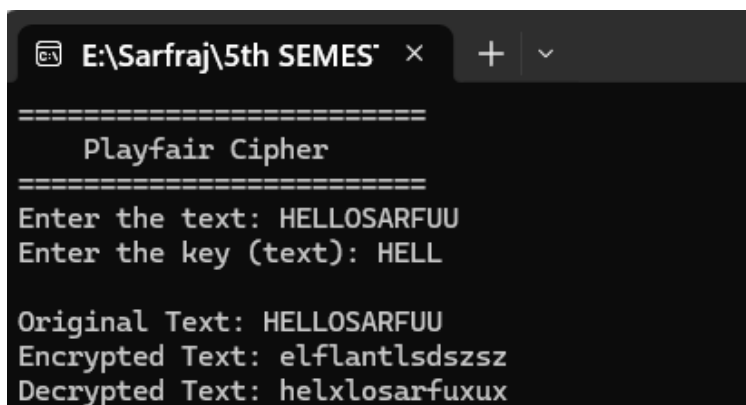
int result_len = 0;
for (int i = 0; i < digraph_len; i += 2) {
    char a = digraphs[i], b = digraphs[i + 1];
    int r1, c1, r2, c2;
    find_position(a, matrix, &r1, &c1);
    find_position(b, matrix, &r2, &c2);
    if (r1 == r2) {
        int shift = encrypt ? 1 : -1;
        result[result_len++] = matrix[r1][(c1 + shift + 5) % 5];
        result[result_len++] = matrix[r2][(c2 + shift + 5) % 5];
    } else if (c1 == c2) {
        int shift = encrypt ? 1 : -1;
        result[result_len++] = matrix[(r1 + shift + 5) % 5][c1];
        result[result_len++] = matrix[(r2 + shift + 5) % 5][c2];
    } else {
        result[result_len++] = matrix[r1][c2];
        result[result_len++] = matrix[r2][c1];
    }
}
result[result_len] = '\0';
}

void playfair_cipher(const char *text, const char *key, int encrypt, char *output) {
    char matrix[5][5];
    generate_matrix(key, matrix);
    process_digraphs(text, matrix, encrypt, output);
}

int main() {
    char text[100], key[100], encrypted[100], decrypted[100];
    printf("=====\n");
    printf("  Playfair Cipher\n");
    printf("=====\n");
    printf("Enter the text: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = '\0';
    printf("Enter the key (text): ");
    scanf("%s", key);
    playfair_cipher(text, key, 1, encrypted); // Encrypt the text
    playfair_cipher(encrypted, key, 0, decrypted); // Decrypt the text
    printf("\nOriginal Text: %s", text);
    printf("\nEncrypted Text: %s", encrypted);
    printf("\nDecrypted Text: %s\n", decrypted);
    return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
=====
  Playfair Cipher
=====
Enter the text: HELLOSARFUU
Enter the key (text): HELL

Original Text: HELLOSARFUU
Encrypted Text: elflantlsdszsz
Decrypted Text: helxlosarfuxux

```

Title: WAP to implement Hill Cipher

Hill Cipher

The Hill Cipher is a polygraphic substitution cipher that encrypts blocks of letters using linear algebra. It employs a key matrix (e.g., 2x2) to transform plaintext into ciphertext via matrix multiplication modulo 26. Each block of plaintext letters is converted to numerical values (A=0, B=1, ..., Z=25), multiplied by the key matrix, and converted back to letters. Decryption uses the inverse key matrix. The key matrix must be invertible modulo 26 (i.e., its determinant must be coprime with 26). Non-alphabetic characters remain unchanged, and padding (e.g., 'x') is added if the text length doesn't match the block size.

Algorithm

1. **Start**
2. **Input Plaintext and Key Matrix**
 - Provide plaintext message and a square invertible key matrix (e.g., 2x2 or 3x3).
3. **Convert Plaintext to Numerical Form**
 - Map letters to numbers: A=0, B=1, ..., Z=25.
4. **Divide Plaintext into Blocks**
 - Split plaintext into blocks of size n (dimension of the key matrix). □
Padding: If the last block is incomplete, add "X" to fill it.
5. **Encrypt Each Block**
 - Multiply each block (as a vector) by the key matrix: $C = K * P \bmod 26$
(C = ciphertext vector, K = key matrix, P = plaintext vector).
6. **Convert Numbers to Ciphertext**
 - Map resulting numerical vectors back to letters.
7. **Decrypt Using Inverse Key Matrix**
 - Compute the modular inverse of the key matrix ($K^{-1} \bmod 26$). □
Multiply ciphertext vectors by K^{-1} :
 $P = K^{-1} * C \bmod 26$.
8. **Convert Numbers to Plaintext**
 - Map decrypted numerical vectors back to letters (remove padding "X" if added).
9. **Stop**

Source Code:

```
#include <stdio.h>
#include <stdlib.h> #include
<ctype.h>
#include <string.h>

// Convert character to position (0-25) or vice versa
int char_to_int(char c) {    return tolower(c) -
'a';
} char int_to_char(int n) {    n = (n % 26 + 26)
% 26; // Ensure n is in 0-25    return 'a' + n;
}

// Compute GCD for checking matrix invertibility
int gcd(int a, int b) {    while (b) {
a %= b;        int temp = a;        a = b;
b = temp;    }    return a;
}

// Find modular multiplicative inverse of a modulo m
int mod_inverse(int a, int m) {    a = a % m;
for (int x = 1; x < m; x++) {        if ((a * x) %
m == 1) {            return x;
        }    }    return -1; //
No inverse exists
}

// Check if 2x2 key matrix is valid (invertible modulo 26) int
is_valid_key(int key[2][2]) {    int det = (key[0][0] * key[1][1]
- key[0][1] * key[1][0]) % 26;    det = (det + 26) % 26; // Ensure
non-negative    return gcd(det, 26) == 1; // Determinant must be
coprime with 26
}

// Compute inverse of 2x2 key matrix modulo 26 void
get_inverse_key(int key[2][2], int inv[2][2]) {    int det =
(key[0][0] * key[1][1] - key[0][1] * key[1][0]) % 26;    det =
(det + 26) % 26; // Ensure non-negative    int det_inv =
mod_inverse(det, 26);
    inv[0][0] = (key[1][1] * det_inv) % 26;
    inv[0][1] = (-key[0][1] * det_inv) % 26;
    inv[1][0] = (-key[1][0] * det_inv) % 26;
    inv[1][1] = (key[0][0] * det_inv) % 26;

    // Ensure all elements are non-negative
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            inv[i][j] = (inv[i][j] + 26) % 26;
        }
    }
}

// Encrypt or Decrypt text using Hill cipher (2x2 matrix) void
hill_cipher(const char *text, int key[2][2], char *result) {
    int len = strlen(text);    char alpha_text[100] = "";    int k
= 0;
```

```

        // Extract alphabetic characters      for
(int i = 0; i < len; i++) {          if
(isalpha(text[i])) {
alpha_text[k++] = tolower(text[i]);
    }
}

// Pad with 'x' if needed
if (k % 2 != 0) {
alpha_text[k++] = 'x';
    }    alpha_text[k] =
'\0';

// Process in pairs      for (int i = 0; i <
k; i += 2) {          int p1 =
char_to_int(alpha_text[i]);          int p2 =
char_to_int(alpha_text[i + 1]);

        // Matrix multiplication: C = K * P mod 26
int c1 = (key[0][0] * p1 + key[0][1] * p2) % 26;          int
c2 = (key[1][0] * p1 + key[1][1] * p2) % 26;
        result[i] = int_to_char(c1);
result[i + 1] = int_to_char(c2);
    }    result[k] =
'\0';
} int main() {    char text[100];
int key[2][2];    int inv_key[2][2];
char encrypted[100], decrypted[100];

printf("=====\n");
printf("    Hill Cipher\n");
printf("=====\n");
    printf("Enter the text: ");    fgets(text, sizeof(text),
stdin);    text[strcspn(text, "\n")] = '\0'; // Remove trailing
newline
    printf("Enter the 2x2 key matrix (4 numbers, row-
wise):\n");    for (int i = 0; i < 2; i++) {          for (int j
= 0; j < 2; j++) {          scanf("%d", &key[i][j]);
    }
}

// Validate key    if (!is_valid_key(key)) {
printf("\nInvalid key! Determinant must be coprime with 26.\n");
return 1;
    }

// Encrypt the text
hill_cipher(text, key, encrypted);

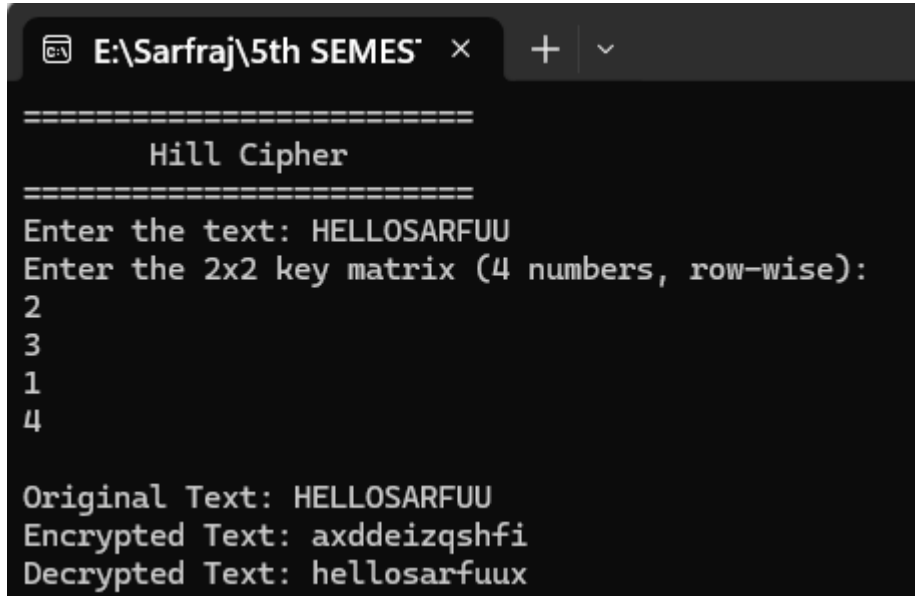
// Compute the inverse key
get_inverse_key(key, inv_key);

// Decrypt the text
hill_cipher(encrypted, inv_key, decrypted);

```

```
        printf("\nOriginal Text: %s", text);  
printf("\nEncrypted Text: %s", encrypted);  
printf("\nDecrypted Text: %s\n", decrypted);  
  
        return 0;  
}
```

Output:



```
E:\Sarfraj\5th SEMES × + ▾  
=====  
Hill Cipher  
=====  
Enter the text: HELLOSARFUU  
Enter the 2x2 key matrix (4 numbers, row-wise):  
2  
3  
1  
4  
  
Original Text: HELLOSARFUU  
Encrypted Text: axddeizqshfi  
Decrypted Text: hellosarfuux
```

Title: WAP to implement Rail fence Cipher

Rail Fence Cipher:

The Rail Fence Cipher is a type of transposition cipher. It works by writing the plaintext in a zigzag pattern (like a rail fence) across multiple "rails" (rows) and then reading the letters row by row to get the ciphertext. It's simple but effective for basic obfuscation. The number of rails is the key, and encryption does not change the characters, only their order.

1. Algorithm:

The Rail Fence Cipher operates as follows:

1. Encryption:

- The plaintext is written in a zigzag pattern across a fixed number of rails (rows), determined by the key.
- The pattern alternates between moving down and up the rails, repeating every $2 * (\text{rails} - 1)$ positions.
- The ciphertext is formed by reading the characters row by row from the resulting grid.

2. Decryption:

- The ciphertext is used to reconstruct the zigzag pattern by determining the positions of characters in the grid based on the number of rails.
- The plaintext is recovered by reading the characters column by column, following the same zigzag path used during encryption.

3. Key Considerations:

- The number of rails must be at least 2 for the cipher to produce a meaningful rearrangement.
- The cipher preserves all characters, including non-alphabetic ones, ensuring the text structure remains intact.
- The security of the cipher depends on the number of rails and the length of the text, though it is relatively weak compared to modern encryption methods due to its predictable pattern.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// Function to encrypt the text using Rail Fence Cipher void
rail_fence_encrypt(const char *text, int rails, char *encrypted) {
    int len = strlen(text);
    char fence[rails][len];
    memset(fence,
0, sizeof(fence)); // Clear the fence array
```

```

        int row = 0, dir =
1;

        // Fill the fence in zigzag pattern
for (int i = 0; i < len; i++) {
fence[row][i] = text[i];        row +=
dir;

        if (row == rails - 1 || row == 0)
{
        dir *= -1; // Change
direction
        }
    }

        // Read the fence row by row    int index =
0;    for (int r = 0; r < rails; r++) {
for (int c = 0; c < len; c++) {        if
(fence[r][c] != 0) {
encrypted[index++] = fence[r][c];
        }
    }    }    encrypted[index] = '\0'; // Null-
terminate the string
}

// Function to decrypt the text using Rail Fence Cipher void
rail_fence_decrypt(const char *encrypted, int rails, char *decrypted) {
int len = strlen(encrypted);    char fence[rails][len];    memset(fence,
0, sizeof(fence)); // Clear the fence array
    int row = 0, dir =
1;

        // Mark positions in the fence
for (int i = 0; i < len; i++) {
fence[row][i] = '*';        row +=
dir;

        if (row == rails - 1 || row == 0)
{
        dir *= -1; // Change
direction
        }
    }

        // Fill the marked positions with the encrypted text
int index = 0;    for (int r = 0; r < rails; r++) {
        for (int c = 0; c < len; c++) {        if
(fence[r][c] == '*' && index < len) {
fence[r][c] = encrypted[index++];
        }
    }
}

        // Read the fence in zigzag order
row = 0;    dir = 1;    for (int i
= 0; i < len; i++) {
decrypted[i] = fence[row][i];
row += dir;

```

```

        if (row == rails - 1 || row == 0)
        {
            dir *= -1; // Change
direction
        }
        } decrypted[len] = '\0'; // Null-
terminate the string
    } int main() {      char text[100], encrypted[100],
decrypted[100];      int rails;

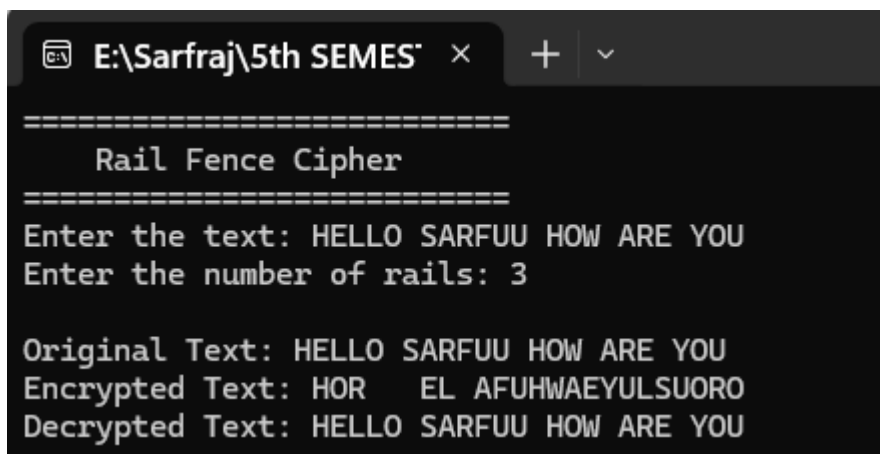
printf("=====\n");
printf("    Rail Fence Cipher\n");
printf("=====\n");
    printf("Enter the text: ");      fgets(text, sizeof(text),
stdin);      text[strcspn(text, "\n")] = '\0'; // Remove trailing
newline
        printf("Enter the number of rails: ");
scanf("%d", &rails);
    if (rails < 2) {          printf("\nInvalid number of rails!
Must be at least 2.\n");      return 1;
    }

    //    Encrypt    and    decrypt    the    text
rail_fence_encrypt(text,      rails,      encrypted);
rail_fence_decrypt(encrypted, rails, decrypted);

    // Display results      printf("\nOriginal
Text: %s\n", text);      printf("Encrypted
Text: %s\n", encrypted);      printf("Decrypted
Text: %s\n", decrypted);
    return
0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
=====
Rail Fence Cipher
=====
Enter the text: HELLO SARFUU HOW ARE YOU
Enter the number of rails: 3

Original Text: HELLO SARFUU HOW ARE YOU
Encrypted Text: HOR EL AFUHWAEYULSUORO
Decrypted Text: HELLO SARFUU HOW ARE YOU

```

Title: WAP to implement Euclidean Algorithm

Euclidean Algorithm

The Euclidean Algorithm is an efficient method for computing the Greatest Common Divisor (GCD) of two integers, a and b . Attributed to the ancient Greek mathematician Euclid, this algorithm is based on the principle that the GCD of two numbers remains the same when the larger number is replaced by its difference with the smaller number. Specifically, it uses the property that $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$, where $\%$ denotes the modulo operation. The algorithm iteratively applies this rule until the remainder is zero, at which point the last non-zero remainder is the GCD. This method is widely used in number theory, particularly in applications like simplifying fractions, solving Diophantine equations, and cryptographic algorithms.

Algorithm

The Euclidean Algorithm operates as follows:

1. Input:

- Take two integers, a and b (positive or negative). ○ If the numbers are negative, convert them to positive, as GCD is always positive.

2. Base Case:

- If b equals 0, the GCD is a .

3. Iterative/Recursive Step:

- Replace a with b and b with $a \% b$.
- Repeat until b becomes 0.

4. Output:

- The final non-zero value of a is the GCD of the original two numbers.

5. Key Considerations:

- The algorithm is highly efficient, with a time complexity of $O(\log(\min(a, b)))$.
- It works for any pair of integers, including zero (where $\text{GCD}(a, 0) = |a|$).
- The GCD is the largest positive integer that divides both input numbers without leaving a remainder.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int euclidean_gcd(int a, int b)
{
    a = abs(a);    b =
abs(b);    while (b != 0) {
int temp = b;        b = a % b;
a = temp;    }    return a;
```

```

} int main() {
int num1, num2;

printf("=====\n");
printf("      Euclidean Algorithm Program\n");
printf("=====\n");
    printf("Enter the first number: ");
scanf("%d", &num1);
    printf("Enter the second number: ");
scanf("%d", &num2);
    int gcd = euclidean_gcd(num1,
num2);
    printf("\nGCD of %d and %d is: %d\n", num1, num2,
gcd);
    return
0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
      Euclidean Algorithm Program
=====
Enter the first number: 16
Enter the second number: 3

GCD of 16 and 3 is: 1

-----
Process exited after 11.99 seconds with return value 0
Press any key to continue . . . |

```


Title: WAP to implement Extended Euclidean Algorithm

Extended Euclidean Algorithm:

The Extended Euclidean Algorithm is an extension of the Euclidean Algorithm that not only computes the Greatest Common Divisor (GCD) of two integers, a and b , but also finds integer coefficients x and y that satisfy Bézout's identity: $ax + by = \gcd(a, b)$. This algorithm, rooted in the work of Euclid and later formalized, is fundamental in number theory and has applications in cryptography, such as finding modular inverses for algorithms like RSA. It builds on the principle of the Euclidean Algorithm, $\gcd(a, b) = \gcd(b, a \% b)$, but tracks additional variables to derive the coefficients x and y . The algorithm ensures that the GCD is the largest positive integer that divides both a and b without a remainder, and the coefficients provide a linear combination equaling the GCD.

Algorithm

The Extended Euclidean Algorithm operates as follows:

1. **Input:**
 - Take two integers, a and b (positive or negative). ○ If negative, convert to positive for GCD calculation, adjusting coefficients as needed.
2. **Base Case:** ○ If b equals 0, the GCD is $|a|$, x is 1 (or -1 if a is negative), and y is 0.
3. **Recursive/Iterative Step:**
 - Compute $\gcd(b, a \% b)$ and coefficients x_1, y_1 for the recursive call.
 - Update coefficients: $x = y_1$, $y = x_1 - (a / b) * y_1$, where $/$ denotes integer division.
 - Repeat until b becomes 0.
4. **Output:**
 - Return the GCD and coefficients x, y satisfying $ax + by = \gcd(a, b)$.
5. **Key Considerations:**
 - The algorithm is efficient with a time complexity of $O(\log(\min(a, b)))$.
 - It handles zero inputs ($\gcd(a, 0) = |a|$) and negative numbers correctly.
 - The coefficients x and y are not unique but satisfy Bézout's identity, enabling applications like modular arithmetic and solving linear Diophantine equations.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```

// Function to compute GCD and Bézout's coefficients using the Extended Euclidean
Algorithm
int extended_euclidean(int a, int b, int* x, int* y) {    int original_a =
a;    a = abs(a);    b = abs(b);
    if (b == 0)
    {
        *x = (original_a < 0) ? -1 : 1;
        *y = 0;
    }
    return a;
}

int x1, y1;    int gcd =
extended_euclidean(b, a % b, &x1, &y1);

*x = y1;
*y = x1 - (a / b) * y1;
return
gcd;
}

int main() {    int
num1, num2, x, y;

printf("=====\n");
printf("    Extended Euclidean Algorithm Program\n");
printf("=====\n");
    printf("Enter the first number: ");
scanf("%d", &num1);
    printf("Enter the second number: ");
scanf("%d", &num2);
    int gcd = extended_euclidean(num1, num2, &x,
&y);
    printf("\nGCD of %d and %d is: %d\n", num1, num2, gcd);    printf("Coefficients
(x, y) for Bézout's identity (%d * x + %d * y = gcd):\n", num1, num2);    printf("x
= %d, y = %d\n", x, y);    printf("Verification: %d * %d + %d * %d = %d\n", num1, x,
num2, y, num1 * x + num2 * y);
    return 0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
Extended Euclidean Algorithm Program
=====
Enter the first number: 16
Enter the second number: 3

GCD of 16 and 3 is: 1
Coefficients (x, y) for B0zout's identity (16 * x + 3 * y = gcd):
x = 1, y = -5
Verification: 16 * 1 + 3 * -5 = 1

-----
Process exited after 5.032 seconds with return value 0
Press any key to continue . . . |

```

Title: WAP to implement Miller-Rabin Primality Test

Miller-Rabin Primality Test

The Miller-Rabin Primality Test is a probabilistic algorithm used to determine whether a given number is prime or composite. Developed by Gary L. Miller in 1976 and refined by Michael O. Rabin in 1980, it leverages properties of modular arithmetic and Fermat's Little Theorem to efficiently test large numbers for primality. Unlike deterministic methods, the Miller-Rabin test is faster but provides a probabilistic result, with the accuracy increasing as the number of iterations (**k**) grows. The algorithm is based on the property that if n is prime, then for a randomly chosen base a (where $1 < a < n$), either $a^d \equiv 1 \pmod{n}$ or $a^{d \cdot 2^r} \equiv -1 \pmod{n}$ for some r , where $n - 1 = d \cdot 2^s$ and d is odd. If these conditions are not met, the number is composite. This test is widely used in cryptographic applications, such as RSA, due to its efficiency for large numbers.

Algorithm

The Miller-Rabin Primality Test operates as follows:

1. Input:

- Take a number n to test for primality and k , the number of iterations.
- Handle edge cases: if $n = 2$, it is prime; if $n < 2$ or n is even, it is composite.

2. Factorization:

- Express $n - 1$ as $d \cdot 2^s$, where d is odd and s is the number of times 2 divides $n - 1$.

3. Testing Loop:

- For k iterations:
 - Select a random base a in the range $[2, n-2]$.
 - Compute $x = a^d \pmod{n}$ using modular exponentiation.
 - If $x = 1$ or $x = n - 1$, continue to the next iteration.
 - For r from 0 to $s-1$, compute $x = x^2 \pmod{n}$. If $x = n - 1$, continue to the next iteration.
 - If none of these conditions are met, n is composite.

4. Output:

- If all iterations pass, n is likely prime.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Function for modular exponentiation: computes (base^exp) % mod
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;    base = base % mod;    while (exp > 0)
    {
        if (exp % 2 == 1) {        result = (result *
base) % mod;
        }        base = (base *
base) % mod;        exp = exp / 2;
    }    return
result;
}

// Function for Miller-Rabin primality test
int miller_rabin_test(long long n, int k) {
    long long d = n - 1;    int s = 0;

    // Handle edge cases    if (n == 2) return 1;
    // Prime    if (n < 2 || n % 2 == 0) return 0; //
Composite

    // Factor n - 1 into d * 2^s, where d is odd
    while (d % 2 == 0) {        d = d / 2;
s = s + 1;
    }

    // Perform k iterations of the test    for (int i = 0; i < k; i++) {
long long a = 2 + rand() % (n - 3); // Random base a in [2, n-2]
long long x = mod_pow(a, d, n);    // Compute a^d % n
        if (x == 1 || x == n - 1)
continue;
        int continueLoop = 0;
        for (int r = 0; r < s - 1; r++) {
x = (x * x) % n;            if (x == n -
1) {                continueLoop = 1;
break;
            }        }        if
(continueLoop) continue;
        return 0; //
Composite
    }    return 1; // Probably
prime
}

int main() {    long long n;    int k;
printf("=====\n");
printf("    Miller-Rabin Primality Test Program\n");
printf("=====\n");
    printf("Enter the number to test for primality:
");    scanf("%lld", &n);
    printf("Enter the number of iterations (e.g., 5 for good accuracy):
");    scanf("%d", &k);
```

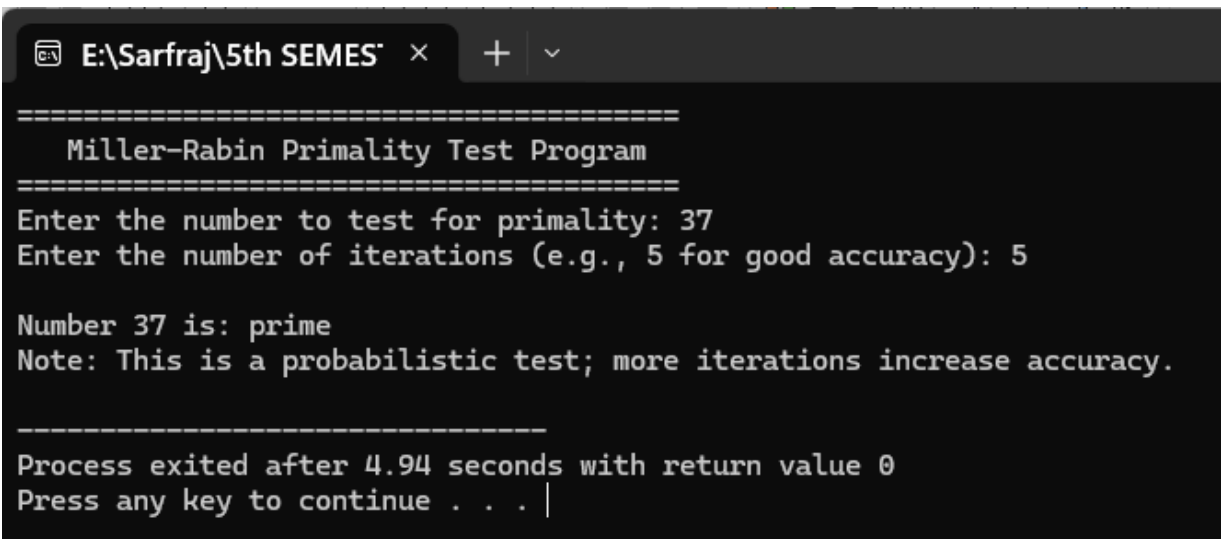
```

        // Seed the random number generator
srand(time(0));
        int is_prime = miller_rabin_test(n,
k);
        printf("\nNumber %lld is: %s\n", n, is_prime ? "prime" : "composite");
printf("Note: This is a probabilistic test; more iterations increase accuracy.\n");

        return 0;
}

```

Output:



```

E:\Sarfraj\5th SEMES x + v
=====
Miller-Rabin Primality Test Program
=====
Enter the number to test for primality: 37
Enter the number of iterations (e.g., 5 for good accuracy): 5

Number 37 is: prime
Note: This is a probabilistic test; more iterations increase accuracy.

-----
Process exited after 4.94 seconds with return value 0
Press any key to continue . . . |

```

Title: WAP to implement Euler's Theorem

Euler's Theorem

Euler's Theorem is a fundamental result in number theory, named after Leonhard Euler, which states that if two integers a and n are coprime (i.e., their greatest common divisor is 1), then a raised to the power of Euler's totient function of n , denoted $\phi(n)$, is congruent to 1 modulo n : $a^{\phi(n)} \equiv 1 \pmod{n}$. The totient function $\phi(n)$ counts the number of integers from 1 to n that are coprime with n . For a prime number p , $\phi(p) = p - 1$, and for a product of distinct primes, $\phi(n)$ is computed as n times the product of $(1 - 1/p_i)$ for each prime factor p_i of n . This theorem is a generalization of Fermat's Little Theorem and is crucial in cryptographic algorithms like RSA, where it underpins modular exponentiation and the computation of modular inverses.

Algorithm

Euler's Theorem is demonstrated as follows:

- 1. Input:** ○ Take two integers: a (the base) and n (the modulus, positive).
- 2. Validation:**
 - Check if $n > 0$; if not, the modulus is invalid.
 - Check if $a > 0$; if not, the base is invalid.
 - Compute $\text{GCD}(a, n)$ using the Euclidean Algorithm; if $\text{GCD} \neq 1$, a and n are not coprime, and the theorem does not apply.
- 3. Compute Euler's Totient Function ($\phi(n)$):**
 - Initialize $\text{result} = n$.
 - For each prime factor p of n (found by checking divisors up to $\sqrt{n}$):
 - If n is divisible by p , divide n by p repeatedly and update result as $\text{result} - \text{result}/p$.
 - If $n > 1$ after the loop, account for the remaining prime factor.
- 4. Modular Exponentiation:**
 - Compute $a^{\phi(n)} \pmod{n}$ using an efficient modular exponentiation algorithm.
- 5. Output:**
 - If a and n are coprime, the result should be 1, verifying Euler's Theorem.
 - Handle error cases (invalid modulus, base, or non-coprime inputs).
- 6. Key Considerations:**

- The algorithm assumes efficient computation of $\phi(n)$, with a time complexity of $O(\sqrt{n})$ for factorization.
- Modular exponentiation is performed in $O(\log \phi(n) * \log n)$ using the square-and-multiply method.
- The theorem is only valid when a and n are coprime, making GCD computation critical.

Source Code:

```
#include <stdio.h>

// Function to compute GCD of two numbers using Euclidean algorithm
int gcd(int a, int b) {
    while (b > 0) {
        a = a % b;
        b = a - b;
        a = a - b;
    }
    return a;
}

// Function to compute Euler's totient function phi(n)
int euler_totient(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) {
                n = n / i;
            }
            result = result - result / i;
        }
    }
    if (n > 1) {
        result = result - result / n;
    }
    return result;
}

// Function for modular exponentiation: computes (base^exp) % mod
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp = exp / 2;
    }
    return result;
}

// Function to demonstrate Euler's Theorem
long long euler_theorem(int a, int n) {
    if (n <= 0) return -1;
    // Invalid modulus
    if (a <= 0) return -2;
    // Invalid base
    if (gcd(a, n) != 1) return -3;
    // a and n not coprime
    int phi = euler_totient(n);
    return mod_pow(a, phi, n);
}

int main() {
    int a, n;
    const char* results[] = {
        "Invalid: modulus must be positive",
        "Invalid: base must be positive",
        "Invalid: a and n must be coprime",
        "1 (Euler's Theorem holds)"
    };
}
```

```

    };
printf("=====\n");
printf("      Euler's Theorem Demonstration Program\n");
printf("=====\n");
    printf("Enter the base (a): ");
scanf("%d", &a);
    printf("Enter the modulus (n): ");
scanf("%d", &n);
    long long computation = euler_theorem(a,
n);    int index = 3; // Default: theorem
holds    if (computation == -1) index = 0;
else if (computation == -2) index = 1;    else
if (computation == -3) index = 2;
    printf("\nResult for a = %d, n = %d: %s\n", a, n, results[index]);    if
(computation >= 0) {        printf("Computed value: %d^phi(%d) mod %d = %lld\n",
a, n, n, computation);
    }    printf("Note: Euler's Theorem applies only if a and n are
coprime.\n");

    return 0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
      Euler's Theorem Demonstration Program
=====
Enter the base (a): 17
Enter the modulus (n): 3

Result for a = 17, n = 3: 1 (Euler's Theorem holds)
Computed value: 17^phi(3) mod 3 = 1
Note: Euler's Theorem applies only if a and n are coprime.

-----
Process exited after 4.283 seconds with return value 0
Press any key to continue . . . |

```


Title: WAP to find the primitive roots of a number

Primitive Roots

In number theory, a primitive root modulo a prime number p is an integer a (where $1 \leq a < p$) such that the powers $a^1, a^2, \dots, a^{(p-1)}$ modulo p generate all integers from 1 to $p-1$ in some order. This implies that the order of a modulo p , defined as the smallest positive integer k such that $a^k \equiv 1 \pmod{p}$, must equal $p-1$. Primitive roots exist only for certain moduli, notably prime numbers, and are significant in cryptographic systems like Diffie-Hellman key exchange and discrete logarithm problems. For a prime p , the number of primitive roots is given by Euler's totient function $\phi(p-1)$, and they are found by testing whether $a^i \pmod{p} \neq 1$ for all i from 1 to $p-2$ and verifying that $a^{(p-1)} \pmod{p} = 1$, per Fermat's Little Theorem.

Algorithm

The process to find primitive roots of a prime number p is as follows:

1. Input: ○ Take a positive integer p and verify it is prime.
2. Validation:
 - If $p \leq 1$, it is invalid.
 - If p is not prime, primitive roots do not exist (except for $p = 2$, where 1 is the primitive root).
 - If $p = 2$, output 1 as the only primitive root.
3. Primitive Root Check:
 - For each integer a from 1 to $p-1$:
 - Compute $a^i \pmod{p}$ for i from 1 to $p-2$ using modular exponentiation.
 - If any $a^i \pmod{p} = 1$, a is not a primitive root.
 - If a passes the check, verify $a^{(p-1)} \pmod{p} = 1$. ▪ If both conditions hold, a is a primitive root.
4. Output:
 - List all primitive roots found or indicate if none exist (unlikely for a prime).
5. Key Considerations:
 - The algorithm uses modular exponentiation for efficiency, with a time complexity of $O(p * \log p * \log(p-1))$ for testing all candidates.
 - Primality testing is performed using trial division up to $\sqrt{p}$, with complexity $O(\sqrt{p})$.
 - The number of primitive roots depends on $\phi(p-1)$, and for large p , testing can be optimized by checking only prime factors of $p-1$.

Source Code:

```
#include <stdio.h>

#include <stdbool.h>

int gcd(int a, int b)
{
    while (b > 0) {
        a = a % b;
        a = a + b;
        b = a - b;
        a = a - b;
    }
    return a;
}

long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2;
    }
    return result;
}

bool is_prime(long long n) {
    if (n == 2) return true;
    if (n < 2 || n % 2 == 0) return false;
    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

void find_primitive_roots(long long p) {
    if (p <= 1) {
        printf("Input must be a positive integer greater than 1.\n");
        return;
    }
    if (!is_prime(p)) {
        printf("Input %lld is not prime. Primitive roots exist only for prime numbers.\n", p);
        return;
    }
    if (p == 2) {
        printf("Primitive root of 2: 1\n");
        return;
    }
}
```

```

        }          int found = 0;

printf("Primitive roots of %lld:\n", p);

for (long long a = 1; a < p; a++) {          int

is_primitive = 1;          for (long long i = 1;

i < p - 1; i++) {          if (mod_pow(a, i,

p) == 1) {          is_primitive = 0;

break;

        }

    }

    if (is_primitive && mod_pow(a, p - 1, p) == 1) {

printf("%lld ", a);          found++;

        }    }    if (found == 0) {          printf("None found

(unexpected for a prime number).\n");

        } else {          printf("\nTotal primitive roots:

%d\n", found);

        }

} int main() {

long long p;

printf("=====\n");

printf("    Primitive Roots Finder Program\n");

printf("=====\n");

    printf("Enter a prime number (p):

");    scanf("%lld", &p);

find_primitive_roots(p);

    printf("Note: A number a is a primitive root of p if a^1, a^2, ..., a^(p-1) mod p

are distinct.\n");

    return

0;

```

}

Output:

```
E:\Sarfraj\5th SEMES' x + v
=====
Primitive Roots Finder Program
=====
Enter a prime number (p): 17
Primitive roots of 17:
3 5 6 7 10 11 12 14
Total primitive roots: 8
Note: A number a is a primitive root of p if  $a^1, a^2, \dots, a^{(p-1)} \bmod p$  are distinct.

-----
Process exited after 6.781 seconds with return value 0
Press any key to continue . . .
```

Title: WAP to compute the discrete logarithm of a number

Discrete Logarithm

The discrete logarithm problem is a cornerstone of number theory and cryptography, particularly in systems like Diffie-Hellman key exchange and ElGamal encryption. Given a prime number p , a primitive root a modulo p , and an integer b (where $1 \leq b < p$), the discrete logarithm is the unique exponent i such that $b \equiv a^i \pmod{p}$, where $0 \leq i < p$. This exponent is denoted as $\text{dlog}_{\{a,p\}}(b) = i$. A primitive root a of p generates all integers from 1 to $p-1$ through its powers $a^1, a^2, \dots, a^{(p-1)} \pmod{p}$, ensuring a unique solution for i . The difficulty of computing discrete logarithms for large primes underpins the security of many cryptographic algorithms, as no efficient general solution exists for large moduli.

Algorithm

The computation of the discrete logarithm operates as follows:

1. Input:

- Take three integers: a (the base, a primitive root), b (the target number), and p (a prime modulus).

2. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Ensure a and b are positive and $b \not\equiv 0 \pmod{p}$.
- Confirm a is a primitive root of p by checking that $a^i \pmod{p} \neq 1$ for i from 1 to $p-2$ and $a^{(p-1)} \pmod{p} = 1$.

3. Computation:

- Use a brute-force approach: for i from 0 to $p-1$, compute $a^i \pmod{p}$ using modular exponentiation.
- If $a^i \pmod{p} = b \pmod{p}$, then i is the discrete logarithm.

4. Output:

- Return the discrete logarithm i if found, or indicate invalid inputs (nonprime p , non-primitive root a , or no solution).

5. Key Considerations:

- The brute-force approach has a time complexity of $O(p * \log p)$ due to modular exponentiation.
- For large p , more efficient algorithms like the Baby-step Giant-step or Pollard's Rho are preferred, though not implemented here.
- The algorithm assumes a is a primitive root, ensuring a unique solution exists for each b in $\{1, 2, \dots, p-1\}$.

Source Code:

```
#include <stdio.h>

long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int gcd(int a, int b) {
    while (b > 0) {
        a = a % b;
        a = a + b;
        b = a - b;
        a = a - b;
    }
    return a;
}

int is_prime(long long n) {
    if (n == 2) return 1;
    if (n < 2 || n % 2 == 0) return 0;
    for (long long i = 3; i * i <= n; i += 2)
        if (n % i == 0) return 0;
    return 1;
}

long long discrete_log(long long a, long long b, long long p) {
    if (p <= 1 || !is_prime(p)) return -1;
    if (a <= 0 || b <= 0) return -1;

    for (long long i = 0; i < p; i++) {
        if (mod_pow(a, i, p) == b % p)
            return i;
    }
    return -1;
}

int main() {
    long long a, b, p;
    const char *result[3] = {
        "invalid: p must be prime, a and b must be positive",
        "invalid: a must be a primitive root of p",
        "computed discrete logarithm"
    };

    printf("=====\n");
    printf("      Discrete Logarithm Calculator Program\n");
    printf("=====\n");
```

```

printf("Enter the base (a, a primitive root): ");
scanf("%lld", &a);

printf("Enter the number (b): ");
scanf("%lld", &b);

printf("Enter the prime modulus (p): ");
scanf("%lld", &p);

long long log_value = discrete_log(a, b, p);
int index = 2;

if (log_value == -1) index = 0;

printf("\nResult for a = %lld, b = %lld, p = %lld: %s\n", a, b, p, result[index]);

if (log_value >= 0) {
    printf("dlog_%lld,%lld(%lld) = %lld\n", a, p, b, log_value);
    printf("Verification: %lld^%lld mod %lld = %lld\n",
           a, log_value, p, mod_pow(a, log_value, p));
}

printf("Note: b = a^i mod p, where i is the discrete logarithm.\n");

return 0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
Discrete Logarithm Calculator Program
=====
Enter the base (a, a primitive root): 23
Enter the number (b): 5
Enter the prime modulus (p): 3

Result for a = 23, b = 5, p = 3: computed discrete logarithm
dlog_23,3(5) = 1
Verification: 23^1 mod 3 = 2
Note: b = a^i mod p, where i is the discrete logarithm.

-----
Process exited after 4.843 seconds with return value 0
Press any key to continue . . . |

```

Title: WAP to implement Diffie-Hellman Key Exchange

Diffie-Hellman Key Exchange

The Diffie-Hellman Key Exchange, introduced by Whitfield Diffie and Martin Hellman in 1976, is a cryptographic protocol that enables two parties, typically referred to as Alice and Bob, to establish a shared secret key over an insecure communication channel. This shared key can then be used for symmetric encryption. The security of the protocol relies on the computational difficulty of the discrete logarithm problem. It uses a publicly agreed-upon prime number p and a primitive root g modulo p . Each party selects a private key (a for Alice, b for Bob) and computes a public key ($g^a \bmod p$ and $g^b \bmod p$, respectively). By exchanging public keys and using their private keys, both parties compute the same shared secret, which is $g^{(a*b)} \bmod p$. The protocol's strength lies in the fact that computing the discrete logarithm to derive the private keys from the public keys is infeasible for large p .

Algorithm

The Diffie-Hellman Key Exchange operates as follows:

1. Input:

- Select a prime number p and a primitive root g modulo p . ○ Alice chooses a private key a , and Bob chooses a private key b (both positive integers).

2. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$). ○ Confirm g is a primitive root of p by ensuring $g^i \bmod p \neq 1$ for i from 1 to $p-2$ and $g^{(p-1)} \bmod p = 1$.
- Ensure private keys a and b are positive.

3. Public Key Computation:

- Alice computes her public key: $A = g^a \bmod p$.
- Bob computes his public key: $B = g^b \bmod p$.

4. Key Exchange:

- Alice and Bob exchange their public keys A and B .

5. Shared Secret Computation:

- Alice computes the shared secret: $S = B^a \bmod p$. ○ Bob computes the shared secret: $S = A^b \bmod p$. ○ Since $B^a \bmod p = (g^b)^a \bmod p = g^{(a*b)} \bmod p = (g^a)^b \bmod p = A^b \bmod p$, both compute the same secret.

6. Output:

- Output the shared secret if valid, or indicate errors (non-prime p , invalid g , or negative private keys).

7. Key Considerations:

- The algorithm's time complexity is $O(\log p * \log(a \text{ or } b))$ per modular exponentiation.
- Security depends on the difficulty of the discrete logarithm problem for large p .
- The protocol assumes a secure channel for public key exchange to prevent man-in-the-middle attacks.

Source Code:

```
#include <stdio.h>
#include <stdbool.h>
int gcd(int a, int b)
{
    while (b > 0) {
        a = a % b;
        a = a + b;
        b = a - b;
        a = a - b;
    }
    return a;
}
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2;
    }
    return result;
}
bool is_prime(long long n) {
    if (n == 2)
        return true;
    if (n < 2 || n % 2 == 0)
        return false;
    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0)
            return false;
    }
    return true;
}
bool is_primitive_root(long long g, long long p) {
    if (p <= 1 || !is_prime(p))
        return false;
    if (gcd(g, p) != 1)
        return false;
    for (long long i = 1; i < p - 1; i++) {
        if (mod_pow(g, i, p) == 1)
            return false;
    }
    return mod_pow(g, p - 1, p) == 1;
}
void diffie_hellman(long long p, long long g, long long a, long long b) {
    if (p <= 1 || !is_prime(p)) {
        printf("Invalid: p must be prime\n");
        return;
    }
    if (!is_primitive_root(g, p)) {
        printf("Invalid: g must be a primitive root of p\n");
        return;
    }
    if (a <= 0 || b <= 0) {
        printf("Invalid: private keys a and b must be positive\n");
        return;
    }
    long long A = mod_pow(g, a, p);
    long long B = mod_pow(g, b, p);
    long long S_alice = mod_pow(B, a, p);
    long long S_bob = mod_pow(A, b, p);
    printf("=====\n");
    printf("Diffie-Hellman Key Exchange\n");
    printf("=====\n");
    printf("Public Prime (p): %lld\n", p);
    printf("Public Primitive Root (g): %lld\n", g);
    printf("Alice's Private Key (a): %lld\n", a);
    printf("Bob's Private Key (b): %lld\n", b);
    printf("Alice's Public Key (A = g^a mod p): %lld\n", A);
    printf("Bob's Public Key (B = g^b mod p): %lld\n", B);
}
```

```

printf("Shared Secret (Alice, B^a mod p): %lld\n", S_alice);
printf("Shared Secret (Bob, A^b mod p): %lld\n", S_bob);
    if (S_alice == S_bob) {        printf("Success: Shared
secret key is %lld\n", S_alice);
    } else {        printf("Error: Shared secrets do
not match\n");
    }
} int main() {        long
long p, g, a, b;

printf("=====\n");
printf("        Diffie-Hellman Key Exchange Program\n");
printf("=====\n");
    printf("Enter the prime number (p): ");
scanf("%lld", &p);
    printf("Enter the primitive root (g):
");    scanf("%lld", &g);
    printf("Enter Alice's private key (a):
");    scanf("%lld", &a);
    printf("Enter Bob's private key (b): ");
scanf("%lld", &b);
    diffie_hellman(p, g, a,
b);
    return
0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
        Diffie-Hellman Key Exchange Program
=====
Enter the prime number (p): 17
Enter the primitive root (g): 5
Enter Alice's private key (a): 45
Enter Bob's private key (b): 86
=====
        Diffie-Hellman Key Exchange
=====
Public Prime (p): 17
Public Primitive Root (g): 5
Alice's Private Key (a): 45
Bob's Private Key (b): 86
Alice's Public Key (A = g^a mod p): 3
Bob's Public Key (B = g^b mod p): 2
Shared Secret (Alice, B^a mod p): 15
Shared Secret (Bob, A^b mod p): 15
Success: Shared secret key is 15
-----
Process exited after 12.42 seconds with return value 0
Press any key to continue . . . |

```

Title: WAP to illustrate Man in Middle Attack

Man-in-the-Middle Attack on Diffie-Hellman Key Exchange

The Man-in-the-Middle (MitM) attack is a significant vulnerability in the basic Diffie-Hellman Key Exchange protocol, which allows two parties, Alice and Bob, to establish a shared secret key over an insecure channel. In an MitM attack, an adversary, often referred to as Eve, intercepts the public keys exchanged between Alice and Bob and substitutes them with her own. This enables Eve to establish separate shared secrets with each party, effectively masquerading as Bob to Alice and as Alice to Bob. As a result, Eve can decrypt, read, modify, and re-encrypt messages, compromising communication while Alice and Bob believe they are communicating securely. The attack exploits the lack of authentication in the basic Diffie-Hellman protocol, as it does not verify the identity of the parties exchanging public keys. The security of the protocol relies on the difficulty of the discrete logarithm problem, but this does not protect against active interception.

Algorithm

The Man-in-the-Middle attack on the Diffie-Hellman Key Exchange operates as follows:

1. Setup:

- Alice and Bob agree on public parameters: a prime number p and a primitive root g modulo p .
- Alice selects a private key a , and Bob selects a private key b . ○ Eve, the attacker, selects two private keys, e_1 (for communication with Bob) and e_2 (for communication with Alice).

2. Public Key Exchange and Interception:

- Alice computes her public key $A = g^a \bmod p$ and sends it to Bob, but Eve intercepts A and sends her own public key $E_2 = g^{e_2} \bmod p$ to Alice.
- Bob computes his public key $B = g^b \bmod p$ and sends it to Alice, but Eve intercepts B and sends her own public key $E_1 = g^{e_1} \bmod p$ to Bob.

3. Shared Secret Computation:

- Alice computes a shared secret with Eve: $S_1 = E_2^a \bmod p = g^{(e_2 * a)} \bmod p$.
- Bob computes a shared secret with Eve: $S_2 = E_1^b \bmod p = g^{(e_1 * b)} \bmod p$.
- Eve computes her shared secret with Alice: $S_{\text{eve_alice}} = A^{e_2} \bmod p = g^{(a * e_2)} \bmod p$ (matches S_1).
- Eve computes her shared secret with Bob: $S_{\text{eve_bob}} = B^{e_1} \bmod p = g^{(b * e_1)} \bmod p$ (matches S_2).

4. Communication:

- Alice encrypts messages using S_1 , which Eve can decrypt using $S_{\text{eve_alice}}$, modify if desired, and re-encrypt using $S_{\text{eve_bob}}$ to send to Bob.
- Similarly, Bob encrypts messages using S_2 , which Eve can decrypt and manipulate.

5. Output:

- Display the public keys, private keys, and shared secrets to demonstrate that Eve has successfully established separate shared secrets with Alice and Bob.

6. Key Considerations:

- The attack requires Eve to actively intercept and replace public keys during the exchange, with a time complexity of $O(\log p)$ per modular exponentiation.
- The basic Diffie-Hellman protocol is vulnerable because it lacks authentication; solutions like digital signatures, certificates, or password-authenticated key agreements (e.g., Station-to-Station protocol) can mitigate this.
- The attack remains undetected unless Alice and Bob compare their shared secrets, which requires a separate secure channel.

Source Code:

```
#include <stdio.h>
#include <math.h>
int gcd(int a, int b)
{
    while (b > 0) {
        a = a % b;
        b = a - a % b;
    }
    return a;
}
long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2;
    }
    return result;
}
int is_prime(long long n) {
    if (n == 2)
        return 1;
    if (n < 2 || n % 2 == 0)
        return 0;
    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
int is_primitive_root(long long g, long long p) {
    if (p <= 1 || !is_prime(p))
        return 0;
    if (gcd(g, p) != 1)
        return 0;
    for (long long i = 1; i < p - 1; i++) {
        if (mod_pow(g, i, p) == 1)
            return 0;
    }
    return mod_pow(g, p - 1, p) == 1;
}
```

```

} void diffie_hellman_mitm(long long p, long long g, long long a, long long b, long
long e1, long long e2) {    if (p <= 1 || !is_prime(p)) {        printf("Invalid:
p must be prime\n");        return;    }    if (!is_primitive_root(g, p)) {
printf("Invalid: g must be a primitive root of p\n");        return;    }    if
(a <= 0 || b <= 0 || e1 <= 0 || e2 <= 0) {        printf("Invalid: private keys a,
b, e1, and e2 must be positive\n");        return;
    }    long long A = mod_pow(g, a, p);
long long B = mod_pow(g, b, p);    long long
E1 = mod_pow(g, e1, p);    long long E2 =
mod_pow(g, e2, p);    long long S1 =
mod_pow(E2, a, p);    long long S2 =
mod_pow(E1, b, p);    long long S_eve_alice =
mod_pow(A, e2, p);    long long S_eve_bob =
mod_pow(B, e1, p);

printf("=====\n");
printf("    Diffie-Hellman Man-In-The-Middle Attack\n");
printf("=====\n");
printf("Public Prime (p): %lld\n", p);    printf("Public Primitive
Root (g): %lld\n", g);    printf("Alice's Private Key (a):
%lld\n", a);    printf("Bob's Private Key (b): %lld\n", b);
printf("Eve's Private Key for Bob (e1): %lld\n", e1);
printf("Eve's Private Key for Alice (e2): %lld\n", e2);
printf("Alice's Public Key (A): %lld\n", A);    printf("Bob's
Public Key (B): %lld\n", B);    printf("Eve's Public Key for Bob
(E1): %lld\n", E1);    printf("Eve's Public Key for Alice (E2):
%lld\n", E2);    printf("Shared Secret (Alice with Eve): %lld\n",
S1);    printf("Shared Secret (Bob with Eve): %lld\n", S2);
printf("Eve's Shared Secret with Alice: %lld\n", S_eve_alice);
printf("Eve's Shared Secret with Bob: %lld\n", S_eve_bob);
    if (S1 == S_eve_alice && S2 == S_eve_bob)
    {
        printf("Success: Eve has established
shared secrets with both Alice and Bob\n");
    } else {        printf("Error: Shared secrets
do not match\n");
    }
}

} int main() {    long long p,
g, a, b, e1, e2;

printf("=====\n");
printf("    Diffie-Hellman Man-In-The-Middle Program\n");
printf("=====\n");
    printf("Enter the prime number (p): ");
scanf("%lld", &p);
    printf("Enter the primitive root (g):
");    scanf("%lld", &g);
    printf("Enter Alice's private key (a):
");    scanf("%lld", &a);
    printf("Enter Bob's private key (b): ");
scanf("%lld", &b);
    printf("Enter Eve's private key for Bob (e1): ");
scanf("%lld", &e1);
    printf("Enter Eve's private key for Alice (e2): ");
scanf("%lld", &e2);
    diffie_hellman_mitm(p, g, a, b, e1,
e2);
    return
0;
}

```

Output:

```
E:\Sarfraj\5th SEMES x + v
Diffie-Hellman Man-In-The-Middle Program
=====
Enter the prime number (p): 23
Enter the primitive root (g): 5
Enter Alice's private key (a): 54
Enter Bob's private key (b): 62
Enter Eve's private key for Bob (e1): 38
Enter Eve's private key for Alice (e2): 55
=====
Diffie-Hellman Man-In-The-Middle Attack
=====
Public Prime (p): 23
Public Primitive Root (g): 5
Alice's Private Key (a): 54
Bob's Private Key (b): 62
Eve's Private Key for Bob (e1): 38
Eve's Private Key for Alice (e2): 55
Alice's Public Key (A): 9
Bob's Public Key (B): 6
Eve's Public Key for Bob (E1): 3
Eve's Public Key for Alice (E2): 22
Shared Secret (Alice with Eve): 1
Shared Secret (Bob with Eve): 2
Eve's Shared Secret with Alice: 1
Eve's Shared Secret with Bob: 2
Success: Eve has established shared secrets with both Alice and Bob

-----
Process exited after 50.78 seconds with return value 0
Press any key to continue . . . |
```

Title: WAP to implement RSA Algorithm

RSA Algorithm

The RSA (Rivest-Shamir-Adleman) algorithm, introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman, is an asymmetric cryptographic system used for secure data transmission. It employs a public key (e, n) for encryption and a private key (d, n) for decryption, relying on the computational difficulty of factoring the product of two large prime numbers. RSA is widely used in secure communication protocols, such as HTTPS and digital signatures. The algorithm generates keys by selecting two large primes p and q , computing the modulus $n = p * q$ and the totient $\phi(n) = (p-1)(q-1)$, choosing a public exponent e coprime with $\phi(n)$, and calculating the private exponent d such that $d * e \equiv 1 \pmod{\phi(n)}$. Encryption transforms a message m into ciphertext $c = m^e \bmod n$, and decryption recovers $m = c^d \bmod n$. The security stems from the impracticality of factoring n for large primes.

Algorithm

The RSA algorithm operates as follows:

1. Key Generation:

- Select two large prime numbers p and q .
- Compute the modulus $n = p * q$.
- Compute Euler's totient function $\phi(n) = (p-1)(q-1)$.
- Choose a public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
- Compute the private exponent d using the modular inverse: $d * e \equiv 1 \pmod{\phi(n)}$.

2. Encryption:

- Take a message m (where $0 \leq m < n$).
- Compute the ciphertext $c = m^e \bmod n$ using modular exponentiation.

3. Decryption:

- Compute the original message $m = c^d \bmod n$ using modular exponentiation.

4. Validation:

- Verify p and q are prime using a primality test (e.g., trial division up to $\sqrt{n}$).
- Ensure e is coprime with $\phi(n)$ using the Euclidean algorithm.
- Check that m is in the range $[0, n)$.
- Confirm d exists by computing the modular inverse of e modulo $\phi(n)$.

5. Output:

- Display the public key (e, n), private key (d, n), ciphertext c, and decrypted message m.
- Verify decryption success if the decrypted message matches the original.

Source Code:

```
#include <stdio.h>
#include <stdbool.h>

long long gcd(long long a, long long b)
{
    while (b != 0) {
        long long
temp = b;
        b = a % b;
        a =
temp;
    }
    return a;
}

bool is_prime(long long n) {
    if (n <= 1)
return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (long
long i = 3; i * i <= n; i += 2) {
        if (n
% i == 0) return false;
    }
    return
true;
}

long long mod_pow(long long base, long long exp, long long
mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
result =
(result * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2;
    }
    return
result;
}

long long mod_inverse(long long e, long long
phi) {
    long long t = 0, new_t = 1;
    long
long r = phi, new_r = e;
    while (new_r != 0) {
        long long
quotient = r / new_r;
        long long
temp_t = t;
        t = new_t;
        new_t =
temp_t - quotient * new_t;
        long long temp_r
= r;
        r = new_r;
        new_r = temp_r -
quotient * new_r;
    }
    if (r > 1)
return -1;
    if (t < 0)
t += phi;
    return t;
}

void rsa(long long p, long long q, long long e, long long m) {
    if (!is_prime(p) || !is_prime(q)) {
        printf("Invalid: p and
q must be prime\n");
        return;
    }
    if (p <= 1 || q <=
1) {
        printf("Invalid: p and q must be greater than 1\n");
        return;
    }
    long long n = p * q;
    long long phi = (p - 1)
* (q - 1);
    if (e <= 1 || e >= phi) {
        printf("Invalid:
e must satisfy 1 < e < phi(n)\n");
        return;
    }
    if
(gcd(e, phi) != 1) {
        printf("Invalid: e must be coprime
with phi(n)\n");
        return;
    }
    if (m < 0 || m >= n) {
        printf("Invalid: message m must be in range [0, n)\n");
        return;
    }
    long long d = mod_inverse(e, phi);
    if (d ==
-1) {
        printf("Invalid: modular inverse for e does not
exist\n");
        return;
    }
    long long c = mod_pow(m, e,
n);
    long long decrypted_m = mod_pow(c, d, n);
    printf("=====\n");
    printf("    RSA
```



```

Encryption and Decryption\n");
printf("=====\n");      printf("Prime p:
%lld\n", p);      printf("Prime q: %lld\n", q);      printf("Modulus
n: %lld\n", n);      printf("Totient phi(n): %lld\n", phi);
printf("Public exponent e: %lld\n", e);      printf("Private
exponent d: %lld\n", d);      printf("Original message m: %lld\n",
m);      printf("Encrypted ciphertext c: %lld\n", c);
printf("Decrypted message: %lld\n", decrypted_m);      if (m ==
decrypted_m)      printf("Success: Decrypted message matches
original\n");      else      printf("Error: Decryption
failed\n");
} int main() {      long long p, q, e, m;
printf("=====\n");
printf("      RSA Algorithm Program\n");
printf("=====\n");
printf("Enter first prime number (p): ");
scanf("%lld", &p);      printf("Enter second
prime number (q): ");      scanf("%lld", &q);
printf("Enter public exponent (e): ");
scanf("%lld", &e);      printf("Enter message to
encrypt (m): ");      scanf("%lld", &m);
rsa(p, q, e, m);
      return 0;
}

```

Output:

```

E:\Sarfraj\5th SEMES x + v
=====
      RSA Algorithm Program
=====
Enter first prime number (p): 17
Enter second prime number (q): 13
Enter public exponent (e): 5
Enter message to encrypt (m): 59
=====
      RSA Encryption and Decryption
=====
Prime p: 17
Prime q: 13
Modulus n: 221
Totient phi(n): 192
Public exponent e: 5
Private exponent d: 77
Original message m: 59
Encrypted ciphertext c: 128
Decrypted message: 59
Success: Decrypted message matches original
=====
Process exited after 6.084 seconds with return value 0
Press any key to continue . . . |

```

ElGamal Cryptographic System

The ElGamal cryptographic system, developed by Taher ElGamal in 1985, is an asymmetric encryption algorithm based on the Diffie-Hellman key exchange. It uses a public key (p, g, h) for encryption and a private key a for decryption, with security rooted in the computational difficulty of the discrete logarithm problem. A large prime p and a primitive root g modulo p are chosen, and the public key h is computed as $g^a \bmod p$, where a is the private key. Encryption of a message m produces a ciphertext pair $(c1, c2)$, and decryption recovers the original message. ElGamal is used in applications like secure key exchange and digital signatures, offering probabilistic encryption due to the random integer k , which enhances security by producing different ciphertexts for the same message.

Algorithm

The ElGamal cryptographic system operates as follows:

1. Key Generation:

- Select a large prime p and a primitive root g modulo p .
- Choose a private key a ($1 \leq a \leq p-2$).
- Compute the public key $h = g^a \bmod p$ using modular exponentiation.

2. Encryption:

- Take a message m ($0 \leq m < p$) and a random integer k ($1 \leq k \leq p-2$).
- Compute $c1 = g^k \bmod p$.
- Compute $c2 = m * h^k \bmod p$.
- Output the ciphertext pair $(c1, c2)$.

3. Decryption:

- Compute $s = c1^a \bmod p$.
- Compute the modular inverse $s^{(-1)} \bmod p$ using the Extended Euclidean Algorithm.
- Recover the message $m = c2 * s^{(-1)} \bmod p$.

4. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Confirm g is a primitive root by ensuring $g^i \bmod p \neq 1$ for i from 1 to $p-2$ and $g^{(p-1)} \bmod p = 1$.
- Ensure a , k , and m are within valid ranges and s has a modular inverse.

5. Output:

- Display the public key (p, g, h) , private key a , random k , ciphertext $(c1, c2)$, and decrypted message.

- Verify decryption success if the decrypted message matches the original.

6. Key Considerations:

- The algorithm's security relies on the discrete logarithm problem, with a time complexity of $O(\log p * \log a)$ per modular exponentiation.
- The random k ensures different ciphertexts for the same message, enhancing security.
- Practical implementations use large primes (e.g., 2048 bits) and secure random number generation for k .

Source Code:

```
#include <stdio.h>

#include <stdbool.h>

long long gcd(long long a, long long b)
{
    while (b != 0) {
        long long
temp = b;        b = a % b;        a =
temp;
    }

return a;
}

bool is_prime(long long n) {
    if (n <= 1)
return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (long
long i = 3; i * i <= n; i += 2) {
        if (n
% i == 0) return false;
    }
    return
true;
}

long long mod_pow(long long base, long long exp, long long
mod) {
    long long result = 1;

    base = base % mod;
    while (exp > 0)
{
        if (exp % 2 == 1)
result = (result * base) % mod;

base = (base * base) % mod;
        exp =
exp / 2;
    }
    return result;
}
```

```

} bool is_primitive_root(long long g, long long
p) {    if (p <= 1 || !is_prime(p)) return false;
if (gcd(g, p) != 1) return false;    long long
phi = p - 1;    for (long long i = 1; i < phi;
i++) {        if (mod_pow(g, i, p) == 1) return
false;
    }    return mod_pow(g, phi, p)
== 1;

} long long mod_inverse(long long a, long long
m) {    long long m0 = m, t, q;    long long x0
= 0, x1 = 1;    if (m == 1) return 0;    while
(a > 1) {        q = a / m;        t = m;
m = a % m;        a = t;        t = x0;
x0 = x1 - q * x0;
        x1 = t;    }
if (x1 < 0) x1 += m0;
return x1;

} void elgamal(long long p, long long g, long long a, long long m, long long
k) {    if (!is_prime(p)) {        printf("Invalid: p must be prime\n");
return;    }    if (!is_primitive_root(g, p)) {        printf("Invalid: g
must be a primitive root of p\n");        return;    }    if (a < 1 || a > p
- 2) {        printf("Invalid: private key a must be in range [1, p-2]\n");
return;    }    if (m < 0 || m >= p) {        printf("Invalid: message m
must be in range [0, p)\n");        return;    }    if (k < 1 || k > p - 2)
{        printf("Invalid: random k must be in range [1, p-2]\n");
return;    }    long long h = mod_pow(g, a, p);    long long c1 = mod_pow(g,
k, p);    long long h_k = mod_pow(h, k, p);    long long c2 = (m * h_k) % p;
    long long s = mod_pow(c1, a, p);    long long s_inv =
mod_inverse(s, p);    if (s_inv == -1) {        printf("Invalid:
modular inverse for s does not exist\n");        return;    }

```

```

long long decrypted_m = (c2 * s_inv) % p;

printf("=====\n");      printf("
ElGamal Encryption and Decryption\n");

printf("=====\n");

printf("Prime p: %lld\n", p);      printf("Primitive root g:
%lld\n", g);      printf("Private key a: %lld\n", a);

printf("Public key h (g^a mod p): %lld\n", h);      printf("Random
k: %lld\n", k);      printf("Original message m: %lld\n", m);

printf("Ciphertext (c1, c2): (%lld, %lld)\n", c1, c2);

printf("Decrypted message: %lld\n", decrypted_m);      if
(decrypted_m == m) {      printf("Success: Decrypted message
matches original\n");

        } else {      printf("Error:
Decryption failed\n");

        }

}

int main() {      long long p, g, a, m, k;

printf("=====\n");

printf("      ElGamal Cryptographic System\n");

printf("=====\n");

printf("Enter prime number (p): ");

        scanf("%lld", &p);      printf("Enter
primitive root (g): ");      scanf("%lld",
&g);      printf("Enter private key (a): ");

        scanf("%lld", &a);      printf("Enter message
to encrypt (m): ");      scanf("%lld", &m);

printf("Enter random integer (k): ");

        scanf("%lld", &k);      elgamal(p, g, a, m,
k);      return 0;

}

```

Output:

```
E:\Sarfraj\5th SEMES x + v
=====
ElGamal Cryptographic System
=====
Enter prime number (p): 23
Enter primitive root (g): 5
Enter private key (a): 6
Enter message to encrypt (m): 8
Enter random integer (k): 9
=====
ElGamal Encryption and Decryption
=====
Prime p: 23
Primitive root g: 5
Private key a: 6
Public key h ( $g^a \bmod p$ ): 8
Random k: 9
Original message m: 8
Ciphertext (c1, c2): (11, 3)
Decrypted message: 8
Success: Decrypted message matches original

-----
Process exited after 9.121 seconds with return value 0
Press any key to continue . . . |
```